# 🏗️ HumanTogether 系统架构总览

十亿级人机共存平台 · 分层架构 · 技术选型

## 一、分层架构全景图

接入层

**用户接入层 — 全球用户访问入口**

- 📱 **移动 App** — Flutter / iOS / Android — 跨平台原生体验
- 🌐 **Web 应用** — Next.js + React — SSR + SEO 优化
- 💬 **第三方平台** — 钉钉 / 企业微信 — Bot 接入
- 🔗 **AI Agent** — MCP 协议接入 — AI Agent 生态

⬇️⬇️⬇️

CDN层

**CDN 智能分发层**

- 🌍 **阿里云 CDN** — 全国 2000+ 边缘节点 — 静态资源 + 动态加速
- ⚡ **DNS 智能解析** — 按地域 / 运营商调度 — DNSPod / HTTPDNS
- 🛡️ **DDoS 防护** — Tbit 级防护能力 — 云防清洗

⬇️⬇️⬇️

网关层

**API 网关 + WAF 安全层**

- 🚦 **Kong / Envoy** — API Gateway — 限流 / 鉴权 / 路由
- 🔒 **WAF 防火墙** — Web 应用防火墙 — SQL注入 / XSS / CC
- 🔑 **统一认证** — JWT / OAuth2 / SSO — 身份 + 权限 + 审计
- 📊 **全链路监控** — APM 全链路追踪 — Jaeger + SkyWalking

⬇️⬇️⬇️

应用层

**微服务应用层 — 12 个核心服务**

- 👤 **用户服务** — 注册/认证/画像 — Go + Gin
- 🤖 **AI 对话服务** — 情感陪伴 / 记忆 — Python + FastAPI
- 💼 **机会服务** — 工作/项目/合作 — Go + Gin
- 🤝 **社区服务** — 动态/问答/互助 — Go + Gin
- 💰 **支付服务** — 交易/结算/分账 — Go + Tron
- 🔔 **消息推送** — PUSH / SMS / 站内信 — Go + 极光
- 🔍 **搜索服务** — 全文 / 向量混合 — Rust + Elasticsearch
- ⭐ **推荐服务** — 个性化推荐 — Python + TensorFlow

⬇️⬇️⬇️

**AI / ML 智能层**

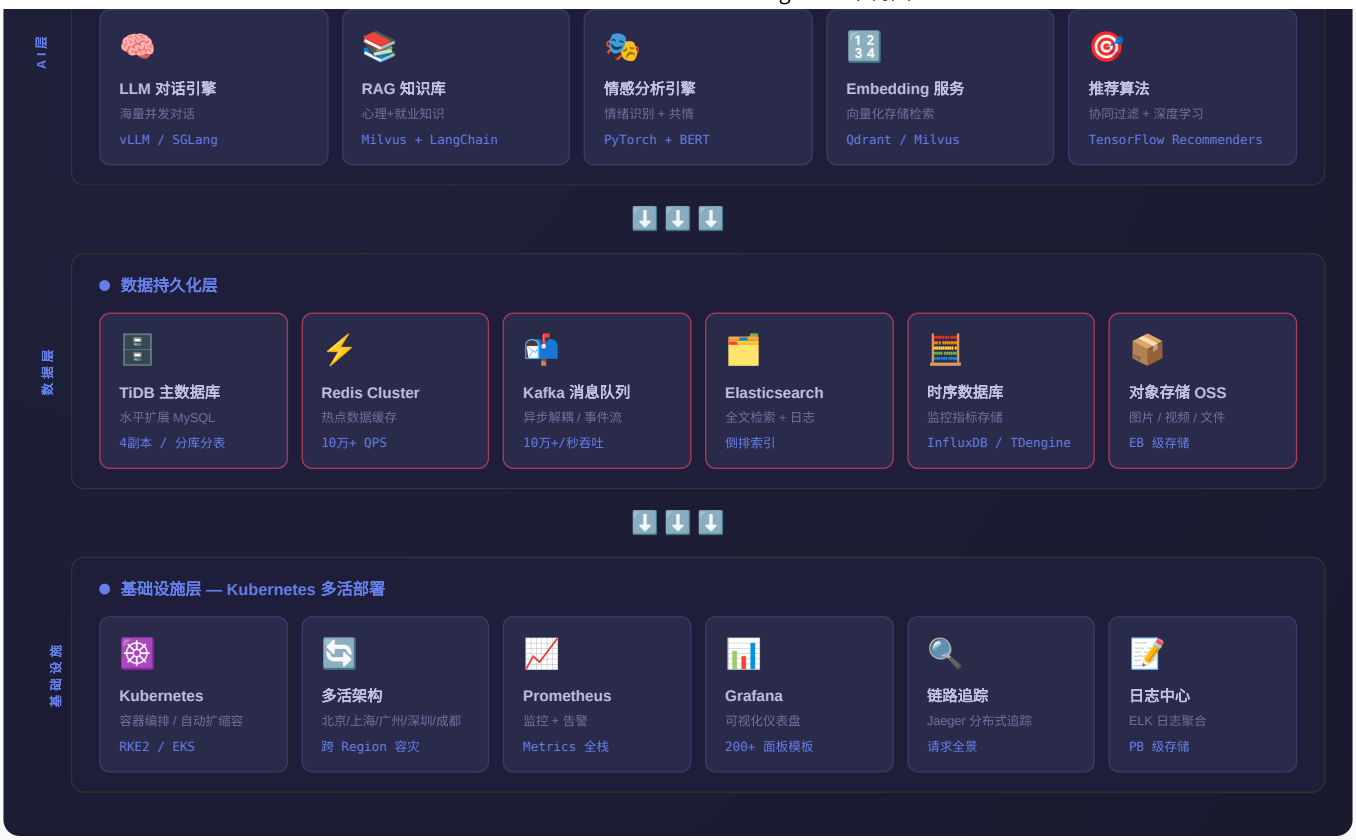



图1：HumanTogether 十亿级分层架构总览 — 从用户接入到基础设施全链路

HumanTogether 技术架构图 · v1.0 · ThinkPad小龙虾 🦞

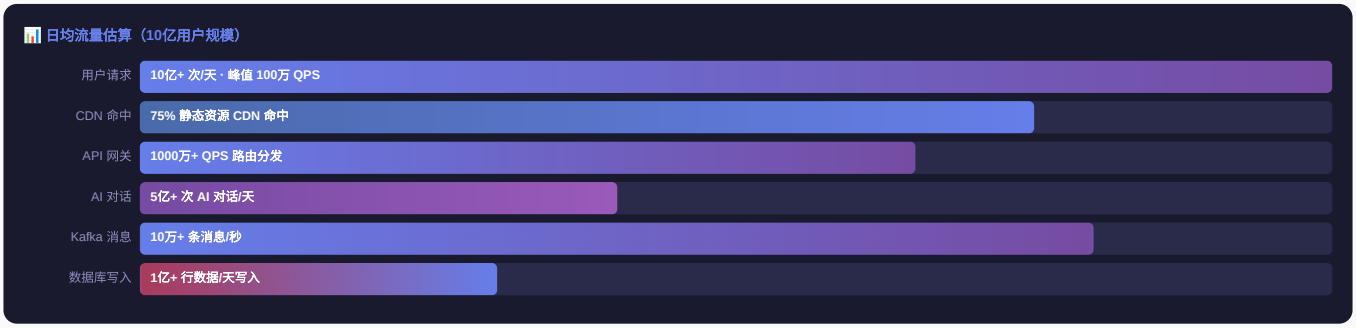
# 🧠 AI 系统架构与数据流

情感陪伴 · RAG 知识库 · 推荐系统 · 对话引擎

## 一、AI 对话系统核心架构

图2：AI 对话系统核心处理流程 — 输入→情感识别→RAG检索→LLM生成→记忆更新

## 二、数据流向 — 全链路追踪




## 三、核心技术指标

| 指标 | Phase 1（100万用户） | Phase 2（1000万用户） | Phase 3（10亿用户） |
|---|---|---|---|
| 同时在线 | 10万 | 100万 | 1亿 |
| API 延迟 P99 | <200ms | <100ms | <50ms |
| AI 对话延迟 | <3s | <1.5s | <500ms |
| 可用性 SLA | 99.5% | 99.9% | 99.99% |

| | | | |
|---|---|---|---|
| 数据存储 | 10TB | 1PB | 10EB |

HumanTogether 技术架构图 · v1.0 · ThinkPad小龙虾 🦞

# 🔧 微服务架构与数据模型

12个核心服务 · 数据库分库分表 · 高可用部署

## 一、12个核心微服务

微服务网格 — 服务间通信 + 服务治理

- 👤 用户服务 UserService — Go · Gin · TiDB — 注册认证·画像·关系链
- 🤖 AI对话服务 AIChat — Python · FastAPI · vLLM — 对话·情感·记忆·RAG
- 💼 机会服务 OpportunitySvc — Go · Gin · TiDB — 工作·项目·合作·悬赏
- 🤝 社区服务 CommunitySvc — Go · Gin · Redis — 动态·问答·互助·师徒
- 💰 支付Pay — Go
- 🔔 推送Push — Go
- 🔍 搜索Search — Rust
- ⭐ 推荐Rec — Python
- 🌙 精神寄托Soul — Go
- 📊 数据Data — Python

🔗 服务间通信

gRPC · HTTP2 高性能调用 | Kafka · 异步事件驱动 | Redis Pub/Sub · 实时通知 | Sentinel · 熔断限流 | Apollo · 配置中心 | Nacos · 服务注册发现

## 二、数据库分库分表策略

### 🗄️ 分库策略

- users 库 → 按 user_id % N
- opportunities 库 → 按 category hash
- messages 库 → 按 room_id hash
- ai_memory 库 → 按 user_id hash
- transactions 库 → 按 user_id

### ⚡ 缓存策略

- L1: 本地缓存 5min（Caffeine）
- L2: Redis Cluster（热数据）
- L3: TiDB（冷数据 + 历史）
- 热点 Key 分散：tag%N 路由
- 缓存预热 + 降级策略

### 🔄 读写分离

- 主库（写）→ 4个从库（读）
- 读请求 80% 路由到从库
- MSR（MySQL Shell）数据校验
- 半同步复制（Semi-sync）
- PingCAP TiDB-operator

## 三、核心数据模型

### 👤 users 用户表

| 字段 | 类型 | 说明 |
|---|---|---|
| id | UUID PK | 用户唯一ID |
| phone | VARCHAR | 手机号（实名） |
| name | VARCHAR | 昵称 |
| identity | ENUM | 学生/职场/自由 |
| skill_tags | JSON | 技能标签数组 |
| auth_level | INT | 实名认证等级 |
| trust_score | DECIMAL | 信任分 0-100 |
| created_at | TIMESTAMP | 注册时间 |

### 🤝 tasks 任务表

| 字段 | 类型 | 说明 |
|---|---|---|
| id | BIGINT PK | 任务ID |
| poster_id | UUID FK | 发布方 |
| worker_id | UUID FK | 承接方 |
| title | VARCHAR | 标题 |
| reward | DECIMAL | 报酬金额 |
| status | ENUM | open/in_progress/completed/paid |
| category | VARCHAR | 分类 |
| location | VARCHAR | 地点 |

### 💼 opportunities 机会表

| 字段 | 类型 | 说明 |
|---|---|---|
| id | BIGINT PK | 机会ID |
| user_id | UUID FK | 发布者 |
| title | VARCHAR | 标题 |
| description | TEXT | 详情 |
| category | VARCHAR | 工作/项目/合作 |
| reward | VARCHAR | 报酬（面议/金额） |
| location | VARCHAR | 线上/城市 |
| views | INT | 浏览量 |

### 🧠 ai_memory AI记忆表

| 字段 | 类型 | 说明 |
|---|---|---|
| id | BIGINT PK | 记录ID |
| user_id | UUID FK | 用户 |
| companion_id | VARCHAR | AI伙伴ID |
| messages | JSON | 对话历史 |
| embedding | VECTOR(1536) | 向量表示 |
| summary | TEXT | 对话摘要 |
| last_active | TIMESTAMP | 最后活跃 |
| updated_at | TIMESTAMP | 更新时间 |

HumanTogether 技术架构图 · v1.0 · ThinkPad小龙虾 🦞

# 部署架构与扩展路径

多活容灾 · Kubernetes · 分阶段扩容 · Phase 1-3 里程碑

## 一、多活部署架构

**五地域多活部署 · 同城双活 + 异地灾备**

| 北京 | 上海 | 广州 | 深圳 | 成都 |
|---|---|---|---|---|
| K8s ×3 Master | K8s ×3 Master | K8s ×3 Master | K8s ×3 Master | K8s ×3 Master |
| 主中心 Primary | 副中心 Secondary | 热备 Hot Standby | 冷备 Cold Standby | 冷备 Cold Standby |
| ✓ 全部服务 | ✓ 全部服务 | ✓ 核心服务 | ✓ AI服务 | ✓ AI服务 |

**跨地域网络**

专线互联 100Gbps · DNS 智能切换 <30s · 数据同步延迟 <100ms · TiDB 分布式 5副本 · Kafka 跨地域复制

## 二、分阶段扩展路径

**PHASE 1**

**0 → 100万用户**

- 单体验证 → 微服务化
- 单机房 → 两地三中心
- MySQL → TiDB 分片
- 第三方 LLM API
- MVP 功能：AI陪伴+机会板

💰 成本：¥200万

**PHASE 2**

**100万 → 1000万用户**

- 全微服务 + K8s
- 五地域多活部署
- LLM 自托管（Qwen/DeepSeek）
- 推荐系统 + 搜索服务
- 支付/会员/企业服务

💰 成本：¥2000万

**PHASE 3**

**1000万 → 10亿用户**

- 全球 CDN + 边缘计算
- 自建 LLM 训练集群
- 多云部署（阿里+腾讯+华为）
- AI Agent 生态系统
- 开放平台（API + MCP）

💰 成本：¥5-50亿

## 三、自动扩缩容策略

**📈 扩容触发条件**

| 指标 | 阈值 | 扩容动作 |
|---|---|---|
| CPU 使用率 | > 70% 持续 3min | 扩容 + 2 Pod |
| 内存使用率 | > 75% 持续 2min | 扩容 + 2 Pod |
| P99 延迟 | > 500ms | 扩容 + 4 Pod |
| 请求队列 | > 1000 | 扩容 + 4 Pod |
| 最小实例数 | 每个服务 3 Pod | |
| 最大实例数 | 每个服务 100 Pod | |

**🛡️ 高可用保障**

| 维度 | 方案 |
|---|---|
| Pod 故障恢复 | K8s 自愈 + 探针检测 |
| 节点故障 | 跨 AZ 分布 + 自动迁移 |
| 数据库故障 | TiDB 自动切换 <30s |
| Redis 故障 | Cluster 自动 failover |
| LLM 服务故障 | 多副本 + 降级策略 |
| Region 故障 | DNS 切换 <60s |

## 四、技术栈全景

| Go | Python | Flutter | Next.js | TiDB | vLLM |
|---|---|---|---|---|---|
| 后端主力语言 | AI/ML 开发 | 跨平台 App | Web 前端 | 分布式数据库 | LLM 推理引擎 |

**Milvus**
向量数据库

**K8s**
容器编排

HumanTogether 技术架构图 · v1.0 · ThinkPad小龙虾 🦞